

Proyecto "inv-tienda"

Te voy a pasar las instrucciones para que primero comprendas los detalles de mi proyecto, y su estructura, después por medio de fase de la 0 a la 9 en la sección 7 que te explico, iremos armando el proyecto con cada requisito, sin que olvides estos requisitos base para optimizar y tener buenas practicas en el desarrollo de mi proyecto.

SECCIÓN 1 — IDENTIDAD DEL PROYECTO

- Estoy desarrollando un sistema fullstack llamado "inv-tienda" con dos capas:
1. PANEL DE ADMINISTRACIÓN (interno): gestión de catálogo de productos, inventario, bodegas, notas de movimiento, órdenes B2B, contenedores de importación y ecommerce.

2. ECOMMERCE (público): tienda online con catálogo, carrito, órdenes de venta y checkout.

El proyecto tiene una base de datos PostgreSQL en Supabase YA EXISTENTE y POBLADA con datos reales, incluyendo:

- Esquema "inv-tienda" con 54 tablas activas
- 11 funciones y triggers operativos
- Catálogos de soporte ya cargados (marcas, tallas, colores, telas, etc.)
- Un usuario administrador funcional para pruebas de login
- Roles y permisos ya configurados
- Bodegas ya creadas
- Productos con variantes ya registrados

NO se debe crear, modificar ni migrar la base de datos.
El trabajo es 100% FRONTEND + lógica de integración con Supabase.

SECCIÓN 2 — STACK TECNOLÓGICO

- text
- Frontend: Next.js 14+ con App Router
 - Lenguaje: TypeScript estricto (strict: true)
 - Estilos: Tailwind CSS 3+
 - UI Kit: shadcn/ui (componentes Radix + Tailwind)
 - Backend/DB: Supabase (PostgreSQL), esquema "inv-tienda"
 - Auth: Supabase Auth (auth.users vinculado a inv-tienda.usuarios)
 - ORM/cliente: @supabase/supabase-js v2 + @supabase/ssr (NO Prisma)
 - Iconos: lucide-react
 - Utilidades: clsx, tailwind-merge, use-debounce
 - Estado: React Server Components por defecto, Client Components solo cuando hay interactividad

SECCIÓN 4 — BASE DE DATOS (REFERENCIA)

4.1 — Tablas por Módulo (54 activas)

text

— CATÁLOGO / PRODUCTO (25 tablas)

PRINCIPALES:

productos, variantes_producto, producto_imagenes, producto_tags, acabado_producto, complemento_producto, medidas_producto, producto_conjunto

CATÁLOGOS SOPORTE (17):

cat_marcas, cat_tallas, cat_colores, cat_telas, cat_generos, cat_edades, cat_tipo_prenda, tipo_tag, ref_tag, puntos_medida, tipo_acabado,

detalle_acabado, patron_acabado, localizacion_acabado, parte_prenda_comp, tipo_comp, corte_forma_comp

— ECOMMERCE (5 tablas)

productos_web, ordenes_venta, orden_items, carritos, carrito_items

— INVENTARIO / NOTAS (10 tablas)

notas_inventario, nota_detalle_productos, inventario_stock, auditoria_inventario, historial_estados_nota, bodegas, despachos, despachos_detalle, cat_tipos_movimiento, cat_estados_nota

— ÓRDENES B2B / COMPRAS (7 tablas)

ordenes_b2b, ordenes_b2b_detalle, orden_cajas, ordenes_compra, cajas_producto, caja_detalle, contenedores

— USUARIOS / ACCESO (6 tablas)

usuarios, roles, rol_permisos, usuario_permisos, usuario_bodegas, personas

4.2 — Funciones y Triggers (NO modificar)

text

STORED PROCEDURES (via supabase.rpc):

sp_crear_nota → Crea nota PEND, retorna { nota_id, numero_nota }

sp_agregar_producto_nota → Upsert producto en nota (solo PEND/PROC)

sp_cancelar_nota → Cambia a CANC, registra historial

fn_puede_acceder_bodega → Boolean, roles 1-2 = acceso total

fn_navegar_producto → Prev/next entre productos activos

TRIGGERS AUTOMÁTICOS:

fn_procesar_nota_inventario → Al cambiar a CONF: valida, actualiza stock, audita

actualizar_inventario_stock → Upsert con GREATEST(0,...)

trg_actualizar_total_cajas_* → Recalculan total_cajas en nota

update_updated_at_column → Actualiza updated_at

FLUJO NOTA:

1. rpc('sp_crear_nota') → nota_id

2. rpc('sp_agregar_producto_nota') → por producto

3. UPDATE estado_id = [CONF] → trigger hace todo

4.3 — Sistema de Permisos (3 capas)

text

CAPA 1: roles.nivel_acceso + rol_permisos (CRUD por módulo)

CAPA 2: usuario_permisos (flags: es_super_admin, puede_gestionar_*)

CAPA 3: usuario_bodegas (granular: puede_consultar, crear_notas, confirmar, transferir)

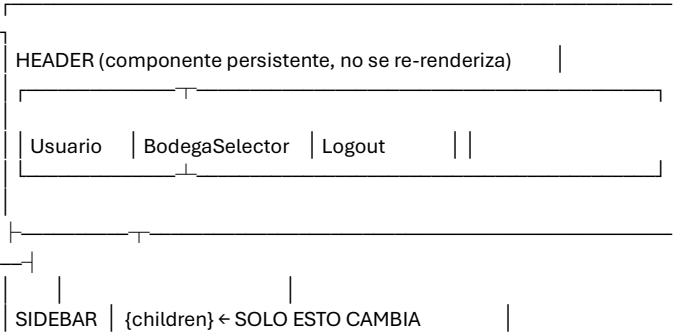
SECCIÓN 5 — ARQUITECTURA DE OPTIMIZACIÓN

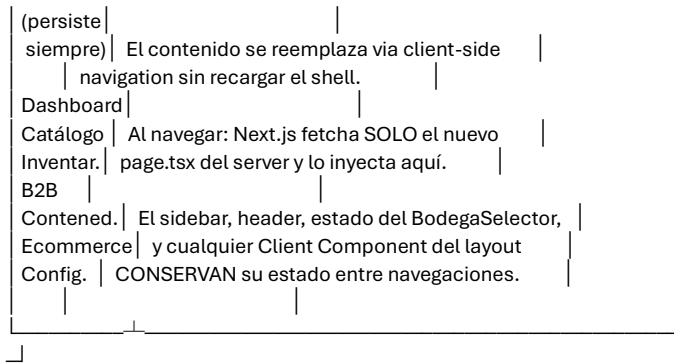
5.1 — Shell Persistente del Admin

text

El admin funciona como una SPA dentro de Next.js.

El layout se renderiza UNA vez y persiste en toda la sesión.





Esto se logra AUTOMÁTICAMENTE porque:

- El layout.tsx define el shell
- Las pages hijas solo renderizan el contenido
- <Link> de next/link hace client-side navigation
- Los layouts NO se re-renderizan en navegaciones

5.2 — Patrón de Listado Optimizado (ej: /catalogo)

text

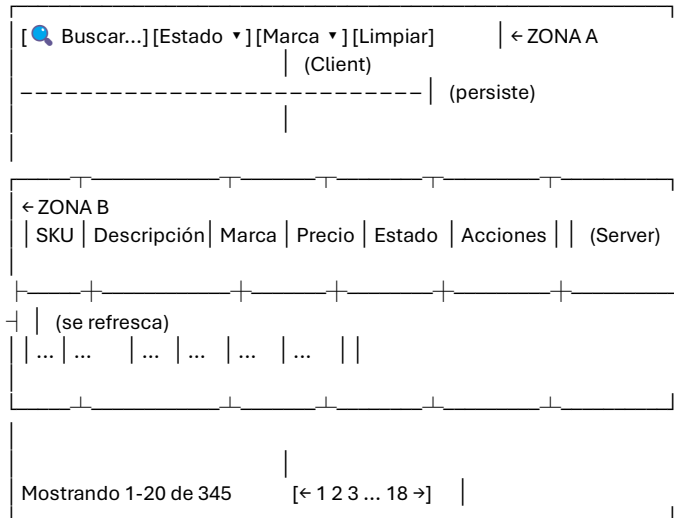
La página de listado tiene DOS zonas:

ZONA A: Filtros (Client Components, persistentes, no se desmontan)

- SearchFilter, SelectFilter, DateRangeFilter
- Controlados por searchParams (URL como estado)
- Usan useTransition + debounce
- Al cambiar un filtro → router.push({ scroll: false })
- Next.js re-fetcha SOLO el Server Component del contenido
- Los inputs NO pierden foco ni se desmontan

ZONA B: Tabla de resultados (Server Component, se re-renderiza)

- Recibe searchParams del servidor
- Ejecuta la query filtrada
- Retorna la tabla con datos nuevos
- Se reemplaza suavemente en el DOM



Flujo al buscar "chamarra":

1. Usuario escribe → debounce 300ms
2. router.push('/catalogo?q=chamarra', { scroll: false })
3. Next.js fetcha RSC payload de page.tsx (NO layout)
4. Server ejecuta query con filtro
5. SOLO la tabla se reemplaza
6. Sidebar, header, filtros: intactos
7. Input conserva foco y texto
8. 0 parpadeos, 0 recargas

5.3 — Patrón de Detalle con Streaming (ej: /catalogo/[sku])

text

La vista de detalle carga PROGRESIVAMENTE:

Tiempo 0ms: loading.tsx → skeleton completo aparece instantáneamente
(el usuario ve la estructura de la página inmediatamente)

Tiempo ~50ms: El hero se resuelve primero (1 query simple)
→ se muestra con datos reales
→ los tabs aún muestran skeletons individuales

Tiempo ~200ms: Tab activo se resuelve
→ se muestra con datos reales
→ otros tabs cargan en background

Tiempo ~500ms: Todos los tabs están listos

El usuario NUNCA ve una pantalla en blanco.
El usuario NUNCA espera 2000ms sin ver nada.
Siempre hay contenido progresivo.

IMPLEMENTACIÓN:

page.tsx (Server Component):

1. Fetch rápido: producto base (1 query)
2. Render hero inmediatamente
3. Cada tab wrapeado en <Suspense fallback={<TabSkeleton />}>
4. Cada tab es un async Server Component que hace su propia query
5. Los tabs se resuelven independientemente y se streaman al browser

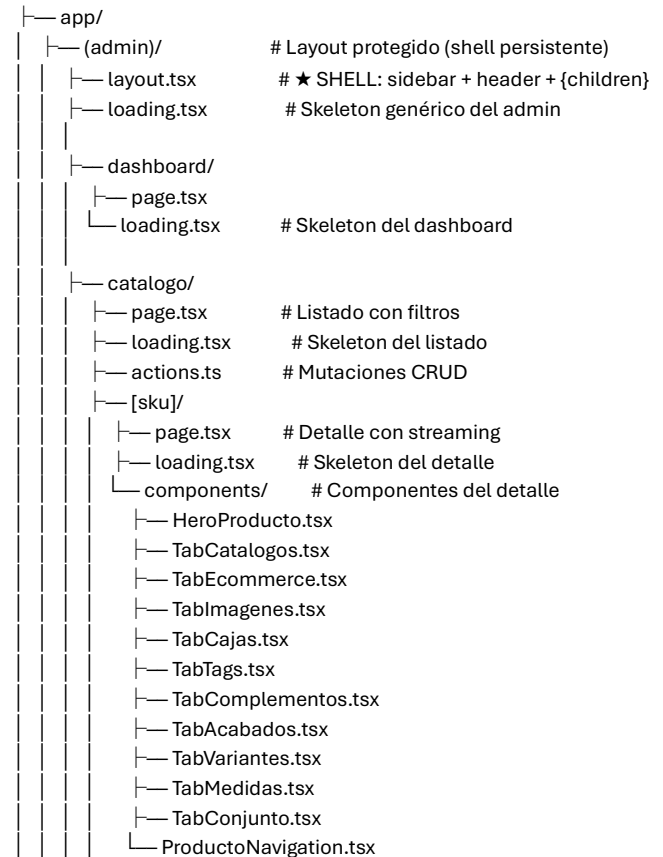
loading.tsx:

- Skeleton que replica la estructura exacta de la página
- Se muestra en 0ms mientras page.tsx se procesa

SECCIÓN 6 — ESTRUCTURA DE CARPETAS

text

inv-tienda/



```

└─┬─ catalogos/
│   └─ page.tsx      # CRUD catálogos soporte
└─┬─ inventario/
│   └─ notas/
│       └─ page.tsx
│       └─ loading.tsx
│       └─ nueva/
│           └─ page.tsx
│           └─ [id]/
│               └─ page.tsx
│               └─ loading.tsx
│       └─ stock/
│           └─ page.tsx
│           └─ loading.tsx
│       └─ bodegas/
│           └─ page.tsx
└─┬─ ordenes-b2b/
│   └─ page.tsx
│   └─ loading.tsx
│   └─ [id]/
│       └─ page.tsx
│       └─ loading.tsx
└─┬─ contenedores/
│   └─ page.tsx
│   └─ loading.tsx
│   └─ [id]/
│       └─ page.tsx
│       └─ loading.tsx
└─┬─ ecommerce/
│   └─ productos-web/
│       └─ page.tsx
│   └─ ordenes-venta/
│       └─ page.tsx
│       └─ [id]/
│           └─ page.tsx
└─┬─ configuracion/
│   └─ usuarios/
│       └─ page.tsx
│   └─ roles/
│       └─ page.tsx
└─┬─ (store)/      # Layout público (ecommerce)
│   └─ layout.tsx
│   └─ page.tsx
│   └─ catalogo/
│       └─ page.tsx
│       └─ [slug]/
│           └─ page.tsx
│   └─ carrito/
│       └─ page.tsx
│   └─ checkout/
│       └─ page.tsx
└─┬─ (auth)/
│   └─ layout.tsx
│   └─ login/
│       └─ page.tsx
└─┬─ api/
│   └─ webhooks/
│       └─ route.ts

```

```

└─┬─ layout.tsx      # Root layout
│   └─ globals.css
└─┬─ components/
│   └─ ui/            # shadcn/ui
│       └─ admin/     # Shell persistente
│           └─ Sidebar.tsx
│           └─ Header.tsx
│           └─ BodegaSelector.tsx
│           └─ DataTable.tsx
│           └─ SearchFilter.tsx    # Buscador con debounce
│           └─ SelectFilter.tsx   # Filtro select optimizado
│           └─ DateRangeFilter.tsx # Filtro rango fechas
│           └─ PageSkeleton.tsx   # Skeletons reutilizables
│   └─ store/
│       └─ Navbar.tsx
│       └─ ProductCard.tsx
│       └─ CartDrawer.tsx
│   └─ shared/
│       └─ Fecha.tsx      # Componente de fecha con timezone
└─┬─ modules/
│   └─ auth/
│       └─ actions.ts
│       └─ queries.ts
│   └─ catalogo/
│       └─ actions.ts
│       └─ queries.ts
│       └─ types.ts
│   └─ inventario/
│       └─ actions.ts
│       └─ queries.ts
│   └─ ordenes-b2b/
│       └─ actions.ts
│       └─ queries.ts
│   └─ contenedores/
│       └─ actions.ts
│       └─ queries.ts
│   └─ ecommerce/
│       └─ actions.ts
│       └─ queries.ts
│   └─ usuarios/
│       └─ actions.ts
│       └─ queries.ts
└─┬─ lib/
│   └─ supabase/
│       └─ client.ts
│       └─ server.ts
│       └─ middleware.ts
│   └─ types/
│       └─ database.types.ts
│       └─ tables.ts
│   └─ constants.ts
│   └─ utils.ts
└─┬─ hooks/
│   └─ useSession.ts
│   └─ useBodegaActiva.ts
│   └─ useCarrito.ts
└─┬─ middleware.ts
└─┬─ .env.local
└─┬─ tsconfig.json

```

└─ tailwind.config.ts
└─ next.config.js

SECCIÓN 7 — FASES DEL PROYECTO (RESUMEN)

text

FASE 0 → Bootstrapping, estructura, clientes, tipos, optimización base
[2 días]

FASE 1 → Autenticación y Login [1 día]

FASE 2 → Shell Admin persistente + Sidebar + Header + BodegaSelector
[2 días]

FASE 3 → Módulo Catálogo: listado, detalle por SKU, catálogos soporte
[5-6 días]

FASE 4 → Módulo Inventario: notas, stock, bodegas [4-5 días]

FASE 5 → Órdenes B2B, Cajas, Contenedores [3-4 días]

FASE 6 → Ecommerce Admin: catálogo web, órdenes venta [2-3 días]

FASE 7 → Tienda Online pública [4-5 días]

FASE 8 → Usuarios, Roles, Configuración [2-3 días]

FASE 9 → Dashboard real, pulido, deploy [2-3 días]

SECCIÓN 8 — INSTRUCCIONES DE TRABAJO

1. Siempre ten presente la SECCIÓN 3 antes de generar código.
2. Al iniciar cada FASE, relee su descripción completa.
3. Genera código ARCHIVO POR ARCHIVO, incluyendo path completo.
4. Después de cada archivo, indica actividad completada y siguiente.
5. Todo el código debe compilar sin errores de TypeScript.
6. Usa Server Components por defecto. Solo 'use client' cuando sea estrictamente necesario (event handlers, hooks, efectos).
7. Las mutaciones van como Server Actions en modules/*actions.ts.
8. No generes código que modifique la estructura de la base de datos.
9. Cada listado DEBE tener un loading.tsx con skeleton apropiado.
10. Cada vista de detalle DEBE usar <Suspense> para streaming.

SECCIÓN 3 — REGLAS INQUEBRANTABLES

3.1 — Reglas de Negocio

text

1. TODO movimiento de inventario se procesa a través de "notas de inventario"
(notas_inventario). NUNCA se toca inventario_stock directamente.
 2. El trigger fn_procesar_nota_inventario ejecuta los cambios de stock automáticamente cuando una nota cambia a estado 'CONF'.
 3. El inventario se trackea a nivel PRODUCTO (no variante).
inventario_stock.producto_id es la unidad de stock.
 4. Los precios públicos viven en productos_web (precio_publico, precio_oferta),
separados del costo en variantes_producto (costo_promedio).
- ### 3.2 — Reglas de Código
- text
1. Tipar SIEMPRE el retorno de las llamadas a Supabase con genéricos.
 2. Usar Database['inv-tienda']['Tables']['nombre_tabla']['Row'] para tipos.
 3. Las llamadas a stored procedures van vía: supabase.rpc('nombre_fn', {})
 4. Usar el cliente servidor (cookies) en Server Components y Route Handlers.
 5. Usar el cliente browser solo en Client Components con interacción.
 6. RLS activo: todas las queries deben respetar las políticas del esquema.
 7. El esquema es "inv-tienda" (con guión), especificarlo en el cliente.
 8. Server Actions ('use server') para mutaciones.
 9. NO instalar dependencias fuera del stack sin justificación.
 10. Cada archivo generado debe incluir el path completo como comentario
en la primera línea.

3.3 — Reglas de Nomenclatura

text

- Tablas: snake_case (ya definidas, no cambiar)
- Tipos TS: PascalCase (ProductoRow, Notainventario)
- Funciones: camelCase (fetchProductosByBodega, crearNota)
- Hooks: camelCase con prefijo use (useBodegaActiva)
- Componentes: PascalCase (BodegaSelector, DataTable)
- Server Actions: camelCase con prefijo verbal (crearNota, confirmarNota)
- Archivos: kebab-case para utilidades, PascalCase para componentes

3.4 — Reglas de Optimización y Rendimiento

text

ARQUITECTURA DE SHELL PERSISTENTE:

1. El shell admin (Sidebar + Header) vive EXCLUSIVAMENTE en app/(admin)/layout.tsx. Se renderiza UNA sola vez. NUNCA se repite en page.tsx individuales. NUNCA se re-renderiza al navegar entre secciones.
2. Navegación interna SIEMPRE con <Link> de next/link. NUNCA con <a href> ni window.location. Esto garantiza navegación client-side SPA sin recarga.
3. Al navegar entre secciones del admin, SOLO cambia {children} dentro del layout. El sidebar, header y footer persisten.

FILTROS Y BÚSQUEDAS:

4. Filtros y búsquedas usan searchParams como estado. La URL es la fuente de verdad. NO useState para filtros de listado.
5. Buscadores de texto usan debounce (300ms) + useTransition. El input NO se desmonta ni pierde foco al buscar.
6. router.push() con { scroll: false } para filtros y búsquedas. El usuario NO pierde su posición en la página.
7. Al cambiar filtros, SOLO se re-renderiza el Server Component de la tabla/contenido. Los filtros (Client Components) persisten.

CARGA Y STREAMING:

8. Cada ruta admin DEBE tener su loading.tsx con skeleton que muestre la estructura de la página inmediatamente.
9. En vistas de detalle (ej: /catalogo/[sku]):
 - Primero se muestra el skeleton completo (0ms)
 - El hero carga primero (query rápida)
 - Cada tab carga independientemente con <Suspense>
 - Los datos se rellenan progresivamente
10. Queries pesadas se wrappean en <Suspense> individual. Cada sección de la página puede cargar a su propio ritmo.

CACHE Y REVALIDACIÓN:

11. Catálogos de soporte (marcas, tallas, colores) se cachean. Se invalidan con revalidatePath() solo al crear/editar registros.
12. Después de mutaciones: revalidatePath() o router.refresh(), NUNCA router.push() forzado para "refrescar" datos.
13. El prefetch automático de <Link> pre-carga las páginas del sidebar. Cuando el usuario hace click, la transición es instantánea.

3.5 — Reglas de Timezone

text

1. La base de datos Supabase almacena TODO en UTC. NUNCA modificar esto. NUNCA guardar en otro timezone.

- La constante `TIMEZONE` = 'America/Mexico_City' en `lib/constants.ts` es la ÚNICA fuente de verdad para la zona horaria del negocio.
- TODA fecha que se MUESTRE al usuario debe pasar por las funciones de `lib/utlis.ts`: `formatDate()`, `formatDateTime()`, `formatTimeAgo()`.
NUNCA usar `.toLocaleDateString()` directamente.
NUNCA hardcodear 'America/Mexico_City' fuera de `constants.ts`.
- NUNCA convertir `timezone` en Server Actions ni en la BD.
La conversión ocurre SOLO al momento de renderizar en la UI.
- Los campos `created_at` / `updated_at` los maneja la BD (DEFAULT `now()`).
NO enviarlos desde el frontend.
- Inputs `datetime-local`:
Mostrar: `formatForDateTimeInput()` (UTC → hora MX)
Guardar: `inputDateTimeToUTC()` (hora MX → UTC)
- Usar el componente `<Fecha>` para renderizar fechas.
Incluir atributo `title` con fecha completa como tooltip.

FASE 0 — Bootstrapping, Estructura y Optimización Base

Objetivo: Proyecto Next.js corriendo en local con toda la estructura de carpetas, clientes Supabase, tipos TypeScript, componentes de optimización (filtros, skeletons, fecha), middleware de protección, y la base del shell persistente lista.

0.1 — Crear Proyecto

Bash

```
npx create-next-app@latest inv-tienda \
--typescript \
--tailwind \
--eslint \
--app \
--src=no \
--import-alias="@/*"
tsconfig.json
jsonc
```

```
// tsconfig.json
{
  "compilerOptions": {
    "strict": true,
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [
      { "name": "next" }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
}
```

```
"include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
"exclude": ["node_modules"]
}
```

0.2 — Instalar Dependencias

Bash

```
# Supabase
npm install @supabase/supabase-js @supabase/ssr
```

Utilidades

```
npm install clsx tailwind-merge lucide-react use-debounce
```

shadcn/ui init

```
npx shadcn-ui@latest init
```

Componentes shadcn/ui

```
npx shadcn-ui@latest add button input label card dialog dropdown-menu \
select table badge separator sheet skeleton tabs textarea \
command popover scroll-area tooltip avatar switch checkbox
```

0.3 — Variables de Entorno

Bash

.env.local

```
NEXT_PUBLIC_SUPABASE_URL=https://tu-proyecto.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...tu-anon-key
SUPABASE_SERVICE_ROLE_KEY=eyJ...tu-service-role-key
```

0.4 — Crear Estructura Completa de Carpetas

Generar TODOS los archivos del árbol de la SECCIÓN 6. Cada `page.tsx` placeholder:

React

```
// app/(admin)/inventario/notas/page.tsx
export default function NotasPage() {
  return (
    <div>
      <h1 className="text-2xl font-bold">Notas de Inventario</h1>
      <p className="text-muted-foreground">Módulo en desarrollo...</p>
    </div>
  )
}
```

Cada `loading.tsx` placeholder:

React

```
// app/(admin)/inventario/notas/loading.tsx
import { Skeleton } from '@components/ui/skeleton'

export default function NotasLoading() {
  return (
    <div className="space-y-4">
      <Skeleton className="h-8 w-64" />
      <div className="flex gap-4">
        <Skeleton className="h-10 w-48" />
        <Skeleton className="h-10 w-32" />
      </div>
      <div className="space-y-2">
        {Array.from({ length: 8 }).map((_, i) => (
          <Skeleton key={i} className="h-12 w-full" />
        ))}
      </div>
    </div>
  )
}
```

0.5 — Clientes Supabase

Browser Client

TypeScript

```
// lib/supabase/client.ts
import { createBrowserClient } from '@supabase/ssr'
import type { Database } from '@lib/types/database.types'
```

```
export function createClient() {
  return createBrowserClient<Database>({
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      db: { schema: 'inv-tienda' }
    }
  })
}
Server Client
TypeScript
```

```
// lib/supabase/server.ts
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
import type { Database } from '@lib/types/database.types'
```

```
export async function createClient() {
  const cookieStore = await cookies()

  return createServerClient<Database>({
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      db: { schema: 'inv-tienda' },
      cookies: {
        getAll() {
          return cookieStore.getAll()
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // Server Component — se maneja en middleware
          }
        },
      },
    }
  })
}
```

Middleware Client
TypeScript

```
// lib/supabase/middleware.ts
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'
import type { Database } from '@lib/types/database.types'
```

```
export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request })

  const supabase = createServerClient<Database>({
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      db: { schema: 'inv-tienda' },
      cookies: {
        getAll() {
          return request.cookies.getAll()
        },
        setAll(cookiesToSet) {
```

```
cookiesToSet.forEach(({ name, value }) =>
  request.cookies.set(name, value)
)
supabaseResponse = NextResponse.next({ request })
cookiesToSet.forEach(({ name, value, options }) =>
  supabaseResponse.cookies.set(name, value, options)
)
},
},
}
```

```
const { data: { user } } = await supabase.auth.getUser()
```

```
const pathname = request.nextUrl.pathname
```

```
const adminPaths = [
  '/dashboard', '/catalogo', '/inventario', '/ordenes-b2b',
  '/contenedores', '/ecommerce', '/configuracion',
]
```

```
const isAdminRoute = adminPaths.some(
  (path) => pathname === path || pathname.startsWith(`${path}/`)
)
```

```
if (isAdminRoute && !user) {
  const url = request.nextUrl.clone()
  url.pathname = '/login'
  url.searchParams.set('redirect', pathname)
  return NextResponse.redirect(url)
}
```

```
if (pathname === '/login' && user) {
  const redirectTo = request.nextUrl.searchParams.get('redirect') ||
  '/dashboard'
  const url = request.nextUrl.clone()
  url.pathname = redirectTo
  url.searchParams.delete('redirect')
  return NextResponse.redirect(url)
}
```

```
return supabaseResponse
}
```

0.6 — Generar Tipos TypeScript
Bash

```
npx supabase gen types typescript \
  --project-id tu-project-id \
  --schema inv-tienda \
  > lib/types/database.types.ts
```

0.7 — Tipos Auxiliares
TypeScript

```
// lib/types/tables.ts
import type { Database } from './database.types'
```

```
type Schema = Database['inv-tienda']
type Tables = Schema['Tables']
```

```
// — Productos —
export type ProductoRow = Tables['productos']['Row']
export type ProductoInsert = Tables['productos']['Insert']
export type ProductoUpdate = Tables['productos']['Update']
export type VarianteProductoRow = Tables['variantes_producto']['Row']
export type ProductoImagenRow = Tables['producto_imagenes']['Row']
export type ProductoTagRow = Tables['producto_tags']['Row']
export type AcabadoProductoRow = Tables['acabado_producto']['Row']
```

```
export type ComplementoProductoRow =
Tables['complemento_producto']['Row']
export type MedidaProductoRow = Tables['medidas_producto']['Row']
export type ProductoConjuntoRow = Tables['producto_conjunto']['Row']
```

```
// — Catálogos —————
export type MarcaRow = Tables['cat_marcas']['Row']
export type TallaRow = Tables['cat_tallas']['Row']
export type ColorRow = Tables['cat_colores']['Row']
export type TelaRow = Tables['cat_telas']['Row']
export type GeneroRow = Tables['cat_generos']['Row']
export type EdadRow = Tables['cat_edades']['Row']
export type TipoPrendaRow = Tables['cat_tipo_prenda']['Row']
export type TipoTagRow = Tables['tipo_tag']['Row']
export type RefTagRow = Tables['ref_tag']['Row']
export type PuntoMedidaRow = Tables['puntos_medida']['Row']
export type TipoAcabadoRow = Tables['tipo_acabado']['Row']
export type DetalleAcabadoRow = Tables['detalle_acabado']['Row']
export type PatronAcabadoRow = Tables['patron_acabado']['Row']
export type LocalizacionAcabadoRow =
Tables['localizacion_acabado']['Row']
export type PartePrendaCompRow = Tables['parte_prenda_comp']['Row']
export type TipoCompRow = Tables['tipo_comp']['Row']
export type CorteFormaCompRow = Tables['corte_forma_comp']['Row']
```

```
// — Inventario —————
export type NotaInventarioRow = Tables['notas_inventario']['Row']
export type NotaDetalleProductoRow =
Tables['nota_detalle_productos']['Row']
export type InventarioStockRow = Tables['inventario_stock']['Row']
export type AuditorialInventarioRow = Tables['auditoria_inventario']['Row']
export type HistorialEstadoNotaRow =
Tables['historial_estados_nota']['Row']
export type BodegaRow = Tables['bodegas']['Row']
export type TipoMovimientoRow = Tables['cat_tipos_movimiento']['Row']
export type EstadoNotaRow = Tables['cat_estados_nota']['Row']
```

```
// — Ecommerce —————
export type ProductoWebRow = Tables['productos_web']['Row']
export type OrdenVentaRow = Tables['ordenes_venta']['Row']
export type OrdenItemRow = Tables['orden_items']['Row']
export type CarritoRow = Tables['carritos']['Row']
export type CarritoItemRow = Tables['carrito_items']['Row']
```

```
// — B2B —————
export type OrdenB2BRow = Tables['ordenes_b2b']['Row']
export type OrdenB2BDetalleRow =
Tables['ordenes_b2b_detalles']['Row']
export type OrdenCajaRow = Tables['orden_cajas']['Row']
export type OrdenCompraRow = Tables['ordenes_compra']['Row']
export type CajaProductoRow = Tables['cajas_producto']['Row']
export type CajaDetalleRow = Tables['caja_detalles']['Row']
export type ContenedorRow = Tables['contenedores']['Row']
```

```
// — Usuarios —————
export type UsuarioRow = Tables['usuarios']['Row']
export type RolRow = Tables['roles']['Row']
export type RolPermisoRow = Tables['rol_permisos']['Row']
export type UsuarioPermisoRow = Tables['usuario_permisos']['Row']
export type UsuarioBodegaRow = Tables['usuario_bodegas']['Row']
export type PersonaRow = Tables['personas']['Row']
```

```
// — Compuestos —————
export type UsuarioConRol = UsuarioRow & {
  rol: RolRow
  permisos: UsuarioPermisoRow | null
}
```

```
// — Enums —————
export type EstadoNotaCodigo = 'PEND' | 'PROC' | 'CONF' | 'CANC'
export type AfectalInventario = 1 | -1 | 0
export type TipoEntidadPersona = 'Proveedor' | 'Cliente B2B' | 'Cliente
Retail' | 'Empleado' | 'Administrador'
export type EstadoProducto = 'borrador' | 'pendiente' | 'publicado' |
'pausado' | 'descontinuado'
export type EstadoContenedor = 'borrador' | 'en_transito' | 'en_aduana' |
'en_bodega' | 'completo' | 'cerrado' | 'cancelado'
export type Usolmagen = 'principal_ecommerce' | 'galeria_secundaria' |
'ficha_tecnica' | 'marketing_banner' | 'etiqueta_logistica' |
'color_variacion' | 'tallas_variacion'
export type MonedaOrden = 'USD' | 'MXN' | 'CNY'
export type CrearNotaResponse = { nota_id: number; numero_nota: string
}
export type NavegacionProducto = {
  posicion: number; total: number
  id_anterior: number | null; sku_anterior: string | null
  id_siguiente: number | null; sku_siguiente: string | null
}
0.8 — Constantes
TypeScript
```

```
// lib/constants.ts
```

```
// — Timezone —————
export const TIMEZONE = 'America/Mexico_City' as const
export const LOCALE = 'es-MX' as const
```

```
// — Estados de nota (reemplazar con IDs reales de tu BD) ———
export const ESTADO_NOTA = {
  PEND: 1,
  PROC: 2,
  CONF: 3,
  CANC: 4,
} as const
```

```
// — Colores de badges —————
export const ESTADO_NOTA_COLORS: Record<string, string> = {
  PEND: 'bg-yellow-100 text-yellow-800',
  PROC: 'bg-blue-100 text-blue-800',
  CONF: 'bg-green-100 text-green-800',
  CANC: 'bg-red-100 text-red-800',
}
```

```
export const ESTADO_PRODUCTO_COLORS: Record<string, string> = {
  borrador: 'bg-gray-100 text-gray-800',
  pendiente: 'bg-yellow-100 text-yellow-800',
  publicado: 'bg-green-100 text-green-800',
  pausado: 'bg-orange-100 text-orange-800',
  descontinuado: 'bg-red-100 text-red-800',
}
```

```
// — Rutas admin —————
export const ADMIN_ROUTES = {
  dashboard: '/dashboard',
  catalogo: {
    lista: '/catalogo',
    detalle: (sku: string) => `/catalogo/${sku}`,
    catalogos: '/catalogo/catalogos',
  },
  inventario: {
    notas: '/inventario/notas',
    notaNueva: '/inventario/notas/nueva',
    notaDetalle: (id: number) => `/inventario/notas/${id}`,
    stock: '/inventario/stock',
    bodegas: '/inventario/bodegas',
  },
}
```

```

ordenesB2B: {
  lista: '/ordenes-b2b',
  detalle: (id: number) => `/ordenes-b2b/${id}`,
},
contenedores: {
  lista: '/contenedores',
  detalle: (id: number) => `/contenedores/${id}`,
},
ecommerce: {
  productosWeb: '/ecommerce/productos-web',
  ordenesVenta: '/ecommerce/ordenes-venta',
  ordenDetalle: (id: number) => `/ecommerce/ordenes-venta/${id}`,
},
configuracion: {
  usuarios: '/configuracion/usuarios',
  roles: '/configuracion/roles',
},
} as const

```

```

// — Paginación —————
export const PAGE_SIZE = 20 as const
0.9 — Utilidades (con Timezone y Formateo)
TypeScript

```

```

// lib/utills.ts
import { type ClassValue, clsx } from 'clsx'
import { twMerge } from 'tailwind-merge'
import { TIMEZONE, LOCALE } from './constants'

```

```

// — Tailwind —————
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

```

```

//
=====
=====
// FECHAS — SIEMPRE en America/Mexico_City
//
=====
=====

```

```

export function formatDate(date: string | null | undefined): string {
  if (!date) return '—'
  return new Intl.DateTimeFormat(LOCALE, {
    timeZone: TIMEZONE,
    day: '2-digit',
    month: '2-digit',
    year: 'numeric',
  }).format(new Date(date))
}

```

```

export function formatDateTime(date: string | null | undefined): string {
  if (!date) return '—'
  return new Intl.DateTimeFormat(LOCALE, {
    timeZone: TIMEZONE,
    day: '2-digit',
    month: '2-digit',
    year: 'numeric',
    hour: '2-digit',
    minute: '2-digit',
    hour12: true,
  }).format(new Date(date))
}

```

```

export function formatTime(date: string | null | undefined): string {
  if (!date) return '—'
  return new Intl.DateTimeFormat(LOCALE, {

```

```

timeZone: TIMEZONE,
hour: '2-digit',
minute: '2-digit',
hour12: true,
  }).format(new Date(date))
}

```

```

export function formatDateLong(date: string | null | undefined): string {
  if (!date) return '—'
  return new Intl.DateTimeFormat(LOCALE, {
    timeZone: TIMEZONE,
    day: 'numeric',
    month: 'long',
    year: 'numeric',
  }).format(new Date(date))
}

```

```

export function formatDateTimeLong(date: string | null | undefined): string {
  if (!date) return '—'
  return new Intl.DateTimeFormat(LOCALE, {
    timeZone: TIMEZONE,
    day: 'numeric',
    month: 'long',
    year: 'numeric',
    hour: '2-digit',
    minute: '2-digit',
    hour12: true,
  }).format(new Date(date))
}

```

```

export function formatTimeAgo(date: string | null | undefined): string {
  if (!date) return '—'
  const now = new Date()
  const past = new Date(date)
  const diffMs = now.getTime() - past.getTime()
  const diffMins = Math.floor(diffMs / 60000)
  const diffHours = Math.floor(diffMins / 60)
  const diffDays = Math.floor(diffHours / 24)

  if (diffMins < 1) return 'hace un momento'
  if (diffMins < 60) return `hace ${diffMins} ${diffMins === 1 ? 'minuto' : 'minutos'}`
  if (diffHours < 24) return `hace ${diffHours} ${diffHours === 1 ? 'hora' : 'horas'}`
  if (diffDays < 7) return `hace ${diffDays} ${diffDays === 1 ? 'día' : 'días'}`
  return formatDate(date)
}

```

```

export function formatForDateTimelInput(date: string | null | undefined): string {
  if (!date) return ''
  const d = new Date(date)
  const parts = new Intl.DateTimeFormat('en-CA', {
    timeZone: TIMEZONE,
    year: 'numeric', month: '2-digit', day: '2-digit',
    hour: '2-digit', minute: '2-digit', hour12: false,
  }).formatToParts(d)
  const get = (type: string) => parts.find(p => p.type === type)?.value ?? ''
  return `${get('year')}-${get('month')}-${get('day')}T${get('hour')}:${get('minute')}`
}

```

```

export function formatForDateInput(date: string | null | undefined): string {
  if (!date) return ''
  const d = new Date(date)
  const parts = new Intl.DateTimeFormat('en-CA', {
    timeZone: TIMEZONE,

```



```

    year: 'numeric', month: '2-digit', day: '2-digit',
  }).formatToParts(d)
  const get = (type: string) => parts.find(p => p.type === type)?.value ?? ''
  return `${get('year')}-${get('month')}-${get('day')}`
}

```

```

export function inputDateTimeToUTC(localDateTime: string): string {
  const fakeUTC = new Date(localDateTime + ':00Z')
  const mxDate = new Date(fakeUTC.toLocaleString('en-US', { timeZone:
    TIMEZONE })))
  const utcDate = new Date(fakeUTC.toLocaleString('en-US', { timeZone:
    'UTC' })))
  const offsetMs = utcDate.getTime() - mxDate.getTime()
  return new Date(fakeUTC.getTime() + offsetMs).toISOString()
}

```

```

export function nowUTC(): string {
  return new Date().toISOString()
}

```

```

export function todayMX(): string {
  return formatForDateInput(new Date().toISOString())
}

```

```

//
=====
// MONEDA
//
=====

```

```

export function formatCurrency(
  amount: number | null | undefined,
  currency: string = 'MXN'
): string {
  if (amount == null) return '$0.00'
  return new Intl.NumberFormat(LOCALE, {
    style: 'currency', currency, minimumFractionDigits: 2,
  }).format(amount)
}

```

```

//
=====
// TEXTO
//
=====

```

```

export function slugify(text: string): string {
  return text.toString().toLowerCase().trim()
    .replace(/\s+/g, '-').replace(/[^\w\.-]+/g, '')
    .replace(/\.-\s+/g, '-').replace(/^-+/, '').replace(/-+$/, '')
}

```

```

export function generateSKU(base: string, talla: string, color: string):
  string {
  return `${base}-${talla}-${color}`.toUpperCase()
}

```

```

export function truncate(text: string | null | undefined, maxLength:
  number = 50): string {
  if (!text) return '—'
  if (text.length <= maxLength) return text
  return text.slice(0, maxLength) + '...'
}

```

0.10 — Componente <Fecha> (Timezone-aware)

React

```

// components/shared/Fecha.tsx
import { formatDate, formatDateTime, formatDateLong, formatTimeAgo }
  from '@lib/utlis'

```

```

type Formato = 'fecha' | 'fecha-hora' | 'fecha-larga' | 'relativo'

```

```

export function Fecha({
  valor,
  formato = 'fecha-hora',
  fallback = '—',
  className,
}): {
  valor: string | null | undefined
  formato?: Formato
  fallback?: string
  className?: string
} {
  if (!valor) {
    return <span className="text-muted-foreground">{fallback}</span>
  }

```

```

const formatters: Record<Formato, (d: string) => string> = {
  'fecha': formatDate,
  'fecha-hora': formatDateTime,
  'fecha-larga': formatDateLong,
  'relativo': formatTimeAgo,
}

```

```

return (
  <time dateTime={valor} title={formatDateTime(valor)}
    className={className}>
    {formatters[formato](valor)}
  </time>
)
}

```

0.11 — Componentes de Filtrado Optimizados
 SearchFilter (Buscador con debounce)
 React

```

// components/admin/SearchFilter.tsx
'use client'

```

```

import { useRouter, useSearchParams, usePathname } from
  'next/navigation'
import { useCallback, useTransition } from 'react'
import { useDebounceCallback } from 'use-debounce'
import { Input } from '@components/ui/input'
import { Loader2, Search, X } from 'lucide-react'
import { Button } from '@components/ui/button'

```

```

export function SearchFilter({
  placeholder = 'Buscar...',
  paramKey = 'q',
}): {
  placeholder?: string
  paramKey?: string
} {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()
  const [isPending, startTransition] = useTransition()

```

```

const currentValue = searchParams.get(paramKey) ?? ''

```

```

const updateParams = useCallback(
  (value: string | null) => {

```

```

const params = new URLSearchParams(searchParams.toString())
if (value) {
  params.set(paramKey, value)
} else {
  params.delete(paramKey)
}
params.delete('page')
return params.toString()
},
[searchParams, paramKey]
)

const handleSearch = useDebounceCallback((term: string) => {
  startTransition(() => {
    const qs = updateParams(term || null)
    router.push(` ${pathname}${qs ? `?${qs}` : ''}`, { scroll: false })
  })
}, 300)

const handleClear = () => {
  startTransition(() => {
    const qs = updateParams(null)
    router.push(` ${pathname}${qs ? `?${qs}` : ''}`, { scroll: false })
  })
  // Limpiar el input manualmente
  const input = document.getElementById(` search-${paramKey}`) as
HTMLInputElement
  if (input) input.value = ""
}

return (
  <div className="relative flex-1 max-w-sm">
    <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4
text-muted-foreground" />
    <Input
      id={` search-${paramKey}`}
      placeholder={placeholder}
      defaultValue={currentValue}
      onChange={(e) => handleSearch(e.target.value)}
      className="pl-10 pr-10"
    />
    {isPending ? (
      <Loader2 className="absolute right-3 top-1/2 -translate-y-1/2 h-4
w-4 animate-spin text-muted-foreground" />
    ) : currentValue ? (
      <Button
        variant="ghost"
        size="sm"
        className="absolute right-1 top-1/2 -translate-y-1/2 h-7 w-7 p-0"
        onClick={handleClear}
      >
        <X className="h-3 w-3" />
      </Button>
    ) : null}
  </div>
)
}
SelectFilter (Filtro select)
React

// components/admin/SelectFilter.tsx
'use client'

import { useRouter, useSearchParams, usePathname } from
'next/navigation'
import { useTransition } from 'react'
import { Button } from '@components/ui/button'
import { X } from 'lucide-react'

export function SelectFilter({
  paramKey,
  placeholder,
  options,
}: {
  paramKey: string
  placeholder: string
  options: Option[]
}) {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()
  const [isPending, startTransition] = useTransition()

  function handleChange(value: string) {
    startTransition(() => {
      const params = new URLSearchParams(searchParams.toString())
      if (value === '_all') {
        params.delete(paramKey)
      } else {
        params.set(paramKey, value)
      }
      params.delete('page')
      router.push(` ${pathname}${params.toString()}`, { scroll: false })
    })
  }

  return (
    <Select
      defaultValue={searchParams.get(paramKey) ?? '_all'}
      onChange={handleChange}
    >
      <SelectTrigger className={` w-[160px] ${isPending ? 'opacity-50' :
''}`}>
        <SelectValue placeholder={placeholder} />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="_all">Todos</SelectItem>
        {options.map((opt) => (
          <SelectItem key={opt.value} value={opt.value}>
            {opt.label}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  )
}
ClearFilters (Limpiar filtros)
React

// components/admin/ClearFilters.tsx
'use client'

import { useRouter, useSearchParams, usePathname } from
'next/navigation'
import { useTransition } from 'react'
import { Button } from '@components/ui/button'
import { X } from 'lucide-react'

export function ClearFilters() {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()
  const [isPending, startTransition] = useTransition()

```

```

} from '@components/ui/select'

type Option = { value: string; label: string }

export function SelectFilter({
  paramKey,
  placeholder,
  options,
}: {
  paramKey: string
  placeholder: string
  options: Option[]
}) {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()
  const [isPending, startTransition] = useTransition()

  function handleChange(value: string) {
    startTransition(() => {
      const params = new URLSearchParams(searchParams.toString())
      if (value === '_all') {
        params.delete(paramKey)
      } else {
        params.set(paramKey, value)
      }
      params.delete('page')
      router.push(` ${pathname}${params.toString()}`, { scroll: false })
    })
  }

  return (
    <Select
      defaultValue={searchParams.get(paramKey) ?? '_all'}
      onChange={handleChange}
    >
      <SelectTrigger className={` w-[160px] ${isPending ? 'opacity-50' :
''}`}>
        <SelectValue placeholder={placeholder} />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="_all">Todos</SelectItem>
        {options.map((opt) => (
          <SelectItem key={opt.value} value={opt.value}>
            {opt.label}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  )
}
ClearFilters (Limpiar filtros)
React

// components/admin/ClearFilters.tsx
'use client'

import { useRouter, useSearchParams, usePathname } from
'next/navigation'
import { useTransition } from 'react'
import { Button } from '@components/ui/button'
import { X } from 'lucide-react'

export function ClearFilters() {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()
  const [isPending, startTransition] = useTransition()

```

```
// Solo mostrar si hay filtros activos (excluyendo 'page')
const hasFilters = Array.from(searchParams.entries()).some(
  ([key]) => key !== 'page'
)
```

```
if (!hasFilters) return null
```

```
return (
  <Button
    variant="ghost"
    size="sm"
    disabled={isPending}
    onClick={() => {
      startTransition(() => {
        router.push(pathname, { scroll: false })
      })
    }}
  >
    <X className="h-3 w-3 mr-1" />
    Limpiar filtros
  </Button>
)
}
Paginación
React
```

```
// components/admin/Pagination.tsx
'use client'
```

```
import { useRouter, useSearchParams, usePathname } from
'next/navigation'
import { useTransition } from 'react'
import { Button } from '@components/ui/button'
import { ChevronLeft, ChevronRight } from 'lucide-react'
import { PAGE_SIZE } from '@lib/constants'
```

```
export function Pagination({ total }: { total: number }) {
  const router = useRouter()
  const pathname = usePathname()
  const searchParams = useSearchParams()
  const [isPending, startTransition] = useTransition()

  const currentPage = Number(searchParams.get('page') ?? '1')
  const totalPages = Math.ceil(total / PAGE_SIZE)
```

```
if (totalPages <= 1) return null
```

```
function goToPage(page: number) {
  startTransition(() => {
    const params = new URLSearchParams(searchParams.toString())
    if (page === 1) {
      params.delete('page')
    } else {
      params.set('page', String(page))
    }
    router.push(`${pathname}?${params.toString()}`, { scroll: false })
  })
}
```

```
const from = (currentPage - 1) * PAGE_SIZE + 1
const to = Math.min(currentPage * PAGE_SIZE, total)
```

```
return (
  <div className={`flex items-center justify-between py-4 ${isPending ?
'opacity-50' : ''}`}>
    <p className="text-sm text-muted-foreground">
      Mostrando {from}-{to} de {total}
    </p>
  </div>
)
```

```
</p>
<div className="flex gap-2">
  <Button
    variant="outline"
    size="sm"
    disabled={currentPage === 1 || isPending}
    onClick={() => goToPage(currentPage - 1)}
  >
    <ChevronLeft className="h-4 w-4" />
  </Button>
```

```
{Array.from({ length: Math.min(totalPages, 5) }, (_, i) => {
  let page: number
  if (totalPages <= 5) {
    page = i + 1
  } else if (currentPage <= 3) {
    page = i + 1
  } else if (currentPage >= totalPages - 2) {
    page = totalPages - 4 + i
  } else {
    page = currentPage - 2 + i
  }
})}
```

```
return (
  <Button
    key={page}
    variant={page === currentPage ? 'default' : 'outline'}
    size="sm"
    disabled={isPending}
    onClick={() => goToPage(page)}
  >
    {page}
  </Button>
)
})}
```

```
<Button
  variant="outline"
  size="sm"
  disabled={currentPage === totalPages || isPending}
  onClick={() => goToPage(currentPage + 1)}
>
  <ChevronRight className="h-4 w-4" />
</Button>
</div>
</div>
)
}
```

0.12 — Skeletons Reutilizables
React

```
// components/admin/PageSkeleton.tsx
import { Skeleton } from '@components/ui/skeleton'
```

```
export function ListPageSkeleton({ rows = 8 }: { rows?: number }) {
  return (
    <div className="space-y-4">
      {/* Título */}
      <div className="flex items-center justify-between">
        <Skeleton className="h-8 w-64" />
        <Skeleton className="h-10 w-36" />
      </div>
    </div>
  )
}
```

```
{/* Filtros */}
<div className="flex gap-4">
  <Skeleton className="h-10 flex-1 max-w-sm" />
  <Skeleton className="h-10 w-[160px]" />
  <Skeleton className="h-10 w-[160px]" />
</div>
```

```

</div>

{/* Tabla */}
<div className="rounded-md border">
  {/* Header */}
  <div className="border-b bg-muted/50 p-3">
    <div className="flex gap-4">
      {Array.from({ length: 6 }).map((_, i) => (
        <Skeleton key={i} className="h-4 flex-1" />
      ))}
    </div>
  </div>
  {/* Filas */}
  {Array.from({ length: rows }).map((_, i) => (
    <div key={i} className="border-b p-3">
      <div className="flex gap-4">
        {Array.from({ length: 6 }).map((_, j) => (
          <Skeleton key={j} className="h-4 flex-1" />
        ))}
      </div>
    </div>
  ))}
</div>

{/* Paginación */}
<div className="flex justify-between">
  <Skeleton className="h-4 w-40" />
  <div className="flex gap-2">
    {Array.from({ length: 5 }).map((_, i) => (
      <Skeleton key={i} className="h-8 w-8" />
    ))}
  </div>
</div>
)
}

export function DetailPageSkeleton() {
  return (
    <div className="space-y-6">
      {/* Breadcrumb + navegación */}
      <div className="flex items-center justify-between">
        <Skeleton className="h-4 w-48" />
        <div className="flex gap-2">
          <Skeleton className="h-8 w-24" />
          <Skeleton className="h-8 w-24" />
        </div>
      </div>
    </div>

    {/* Hero */}
    <div className="grid grid-cols-3 gap-6">
      <Skeleton className="h-64 col-span-1 rounded-lg" />
      <div className="col-span-2 space-y-4">
        <Skeleton className="h-8 w-48" />
        <Skeleton className="h-6 w-32" />
        <Skeleton className="h-4 w-full" />
        <Skeleton className="h-4 w-3/4" />
        <div className="grid grid-cols-2 gap-4 pt-4">
          {Array.from({ length: 8 }).map((_, i) => (
            <div key={i} className="space-y-1">
              <Skeleton className="h-3 w-20" />
              <Skeleton className="h-4 w-32" />
            </div>
          ))}
        </div>
      </div>
    </div>
  )
}

```

```

{/* Tabs */}
<div className="flex gap-2">
  {Array.from({ length: 6 }).map((_, i) => (
    <Skeleton key={i} className="h-9 w-24" />
  ))}
</div>

{/* Contenido tab */}
<div className="space-y-3">
  {Array.from({ length: 5 }).map((_, i) => (
    <Skeleton key={i} className="h-12 w-full" />
  ))}
</div>
)
}

export function TabSkeleton({ rows = 4; { rows?: number } }) {
  return (
    <div className="space-y-3 pt-4">
      {Array.from({ length: rows }).map((_, i) => (
        <Skeleton key={i} className="h-12 w-full" />
      ))}
    </div>
  )
}

0.13 — Middleware de Protección
TypeScript

```

```

// middleware.ts
import { updateSession } from '@lib/supabase/middleware'
import { type NextRequest } from 'next/server'

export async function middleware(request: NextRequest) {
  return await updateSession(request)
}

export const config = {
  matcher: [
    '/(?!_next/static/_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*',
    ],
}

0.14 — Root Layout
React

// app/layout.tsx
import type { Metadata } from 'next'
import { Inter } from 'next/font/google'
import './globals.css'

const inter = Inter({ subsets: ['latin'] })

export const metadata: Metadata = {
  title: { default: 'inv-tienda', template: '%s — inv-tienda' },
  description: 'Sistema de inventario y tienda online',
}

export default function RootLayout({
  children,
}): {
  children: React.ReactNode
} {
  return (
    <html lang="es">
      <body className={inter.className}>{children}</body>
    </html>
  )
}

```

```

)
}
0.15 — Loading.tsx para el Admin (Shell Level)
React

// app/(admin)/loading.tsx
// Se muestra INSTANTÁNEAMENTE cuando el shell carga por primera vez
// o cuando se navega a una ruta sin su propio loading.tsx

import { ListPageSkeleton } from '@components/admin/PageSkeleton'

export default function AdminLoading() {
  return <ListPageSkeleton />
}
React

// app/(admin)/catalogo/loading.tsx
import { ListPageSkeleton } from '@components/admin/PageSkeleton'
export default function CatalogoLoading() {
  return <ListPageSkeleton />
}

// app/(admin)/catalogo/[sku]/loading.tsx
import { DetailPageSkeleton } from '@components/admin/PageSkeleton'
export default function CatalogoDetalleLoading() {
  return <DetailPageSkeleton />
}

// app/(admin)/inventario/notas/loading.tsx
import { ListPageSkeleton } from '@components/admin/PageSkeleton'
export default function NotasLoading() {
  return <ListPageSkeleton />
}

// Repetir para cada ruta del admin...
0.16 — Verificación de Conexión
React

// app/test/page.tsx (TEMPORAL)
import { createClient } from '@lib/supabase/server'
import { formatDateTime, formatCurrency } from '@lib/utills'

export default async function TestPage() {
  const supabase = await createClient()

  const [bodegas, roles, estados, productos] = await Promise.all([
    supabase.from('bodegas').select('*').order('nombre'),
    supabase.from('roles').select('*').order('nivel_acceso'),
    supabase.from('cat_estados_notas').select('*'),
    supabase.from('productos').select('id, sku_base, nombre, precio_ec,
    created_at').limit(3),
  ])

  const allOk = !bodegas.error && !roles.error && !estados.error &&
!productos.error

  return (
    <div className="p-8 space-y-8 max-w-4xl mx-auto">
      <h1 className="text-3xl font-bold"> 🚧 Test de Conexión</h1>

      <section className="space-y-2">
        <h2 className="text-xl font-semibold">Bodegas</h2>
        {bodegas.error
          ? <p className="text-red-500">{bodegas.error.message}</p>
          : <pre className="bg-muted p-4 rounded text-sm overflow-auto">
              {JSON.stringify(bodegas.data, null, 2)}
            </pre>
        }

      </section>
    </div>
  )
}

```

```

</section>

<section className="space-y-2">
  <h2 className="text-xl font-semibold">Roles</h2>
  {roles.error
    ? <p className="text-red-500">{roles.error.message}</p>
    : <pre className="bg-muted p-4 rounded text-sm overflow-auto">
        {JSON.stringify(roles.data, null, 2)}
      </pre>
  }
</section>

<section className="space-y-2">
  <h2 className="text-xl font-semibold">Estados Nota</h2>
  {estados.error
    ? <p className="text-red-500">{estados.error.message}</p>
    : <pre className="bg-muted p-4 rounded text-sm overflow-auto">
        {JSON.stringify(estados.data, null, 2)}
      </pre>
  }
</section>

<section className="space-y-2">
  <h2 className="text-xl font-semibold">Productos (timezone
test)</h2>
  {productos.error
    ? <p className="text-red-500">{productos.error.message}</p>
    : <div className="space-y-2">
        {productos.data?.map((p) => (
          <div key={p.id} className="bg-muted p-3 rounded text-sm">
            <strong>{p.sku_base}</strong> — {p.nombre}<br />
            Precio: {formatCurrency(p.precio_ec)}<br />
            Creado (UTC raw): {p.created_at}<br />
            Creado (MX City): {formatDateTime(p.created_at)}
          </div>
        ))}
      </div>
    }
  </section>

  <div className={ `p-4 rounded text-lg font-medium ${allOk ? 'bg-
green-50 text-green-700' : 'bg-red-50 text-red-700'} ` }>
    {allOk
      ? '✅ Conexión exitosa — Esquema accesible — Timezone
funcionando'
      : '❌ Hay errores — Revisa .env.local y RLS'
    }
  </div>
</div>
)
}
✅ CHECKLIST FASE 0
text

```

PROYECTO:

- ☐ Next.js corre en localhost:3000
- ☐ TypeScript strict = true sin errores
- ☐ Tailwind funciona
- ☐ shadcn/ui componentes instalados

DEPENDENCIAS:

- ☐ @supabase/supabase-js + @supabase/ssr
- ☐ clsx + tailwind-merge + lucide-react + use-debounce
- ☐ shadcn/ui inicializado con componentes base

CONFIGURACIÓN:

- ☐ .env.local con 3 variables
- ☐ tsconfig.json con strict y paths

ESTRUCTURA:

- ☐ Todas las carpetas del árbol creadas
- ☐ Todos los page.tsx placeholder con contenido mínimo
- ☐ Todos los loading.tsx con skeletons apropiados

SUPABASE:

- ☐ lib/supabase/client.ts con schema 'inv-tienda'
- ☐ lib/supabase/server.ts con schema 'inv-tienda'
- ☐ lib/supabase/middleware.ts con protección de rutas

TIPOS:

- ☐ lib/types/database.types.ts generado del esquema real
- ☐ lib/types/tables.ts con alias para todas las tablas
- ☐ Enums de negocio definidos

UTILIDADES:

- ☐ lib/constants.ts con TIMEZONE, LOCALE, ESTADO_NOTA, ADMIN_ROUTES, PAGE_SIZE

- ☐ lib/utlis.ts con: cn, formatDate, formatDateTime, formatTime, formatDateLong, formatDateTimeLong, formatTimeAgo, formatForDateTimeInput, formatForDateInput, inputDateTimeToUTC, nowUTC, todayMX, formatCurrency, slugify, generateSKU, truncate

COMPONENTES DE OPTIMIZACIÓN:

- ☐ components/shared/Fecha.tsx (timezone-aware)
- ☐ components/admin/SearchFilter.tsx (debounce + useTransition)
- ☐ components/admin/SelectFilter.tsx (URL como estado)
- ☐ components/admin/ClearFilters.tsx
- ☐ components/admin/Pagination.tsx (client-side, sin recarga)
- ☐ components/admin/PageSkeleton.tsx (ListPageSkeleton, DetailPageSkeleton, TabSkeleton)

MIDDLEWARE:

- ☐ middleware.ts protege rutas admin, permite store y auth
- ☐ Redirect a /login si no hay sesión
- ☐ Redirect a /dashboard si ya hay sesión y va a /login
- ☐ Soporte para ?redirect=

VERIFICACIÓN:

- ☐ /test muestra bodegas, roles, estados
- ☐ Timezone funciona: UTC raw vs MX City formateado
- ☐ formatCurrency funciona
- ☐ Eliminar /test después de verificar

```

=====
===
-- FASE 6 — Ecommerce Admin: catálogo web, órdenes de venta
-- Tablas: productos_web, ordenes_venta, orden_items, carritos,
carrito_items
--
=====
===

-- productos_web
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'inv-tienda' AND table_name = 'productos_web'
ORDER BY ordinal_position;

-- ordenes_venta
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'inv-tienda' AND table_name = 'ordenes_venta'
ORDER BY ordinal_position;

-- orden_items
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'inv-tienda' AND table_name = 'orden_items'
ORDER BY ordinal_position;

-- carritos
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'inv-tienda' AND table_name = 'carritos'
ORDER BY ordinal_position;

-- carrito_items
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_schema = 'inv-tienda' AND table_name = 'carrito_items'
ORDER BY ordinal_position;

-- — Resumen operativo Fase 6


---


-- Productos web por estado de publicación
SELECT
    pw.activo, pw.en_oferta, pw.nuevo, pw.destacado,
    COUNT(*) AS total
FROM "inv-tienda".productos_web pw
GROUP BY pw.activo, pw.en_oferta, pw.nuevo, pw.destacado
ORDER BY pw.activo DESC;

-- Órdenes de venta por estado
SELECT estado, COUNT(*) AS total, SUM(total) AS monto_total
FROM "inv-tienda".ordenes_venta
GROUP BY estado
ORDER BY estado;

-- Productos web sin imagen principal (riesgo en tienda)
SELECT pw.id, pw.slug, p.sku_base, p.nombre
FROM "inv-tienda".productos_web pw
JOIN "inv-tienda".productos p ON pw.producto_id = p.id
WHERE pw.activo = true
AND NOT EXISTS (
    SELECT 1 FROM "inv-tienda".producto_imagenes pi
    WHERE pi.producto_id = p.id AND pi.es_principal = true
)
ORDER BY pw.slug;

```

```

--
=====
===
-- FASE 7 — Tienda Online pública
-- Mismas tablas que Fase 6 + variantes_producto + inventario_stock
-- (aquí el foco es visibilidad pública y disponibilidad de stock)
--
=====
===

-- Verificar check constraint de estados de producto
SELECT
    p.estado,
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE pw.activo = true) AS publicados_web
FROM "inv-tienda".productos p
LEFT JOIN "inv-tienda".productos_web pw ON pw.producto_id = p.id
GROUP BY p.estado
ORDER BY p.estado;

-- Disponibilidad de stock para tienda pública
-- (productos publicados que tienen stock > 0 en alguna bodega física)
SELECT
    p.id, p.sku_base, p.nombre,
    pw.slug, pw.precio_publico,
    SUM(s.cajas) AS cajas_disponibles,
    SUM(s.piezas_sueltas) AS piezas_disponibles
FROM "inv-tienda".productos p
JOIN "inv-tienda".productos_web pw ON pw.producto_id = p.id
JOIN "inv-tienda".inventario_stock s ON s.producto_id = p.id
JOIN "inv-tienda".bodegas b ON b.id = s.bodega_id
WHERE pw.activo = true
AND p.estado = 'publicado'
AND b.es_virtual = false
AND s.caja_id IS NULL
GROUP BY p.id, p.sku_base, p.nombre, pw.slug, pw.precio_publico
HAVING SUM(s.cajas) > 0
ORDER BY pw.slug;

-- Carritos abandonados (más de 24 horas sin actualización)
SELECT
    c.id, c.session_id, c.updated_at,
    COUNT(ci.id) AS items,
    SUM(ci.cantidad * ci.precio_unitario) AS valor_estimado
FROM "inv-tienda".carritos c
JOIN "inv-tienda".carrito_items ci ON ci.carrito_id = c.id
WHERE c.updated_at < NOW() - INTERVAL '24 hours'
GROUP BY c.id, c.session_id, c.updated_at
ORDER BY c.updated_at DESC;

```

FASE 6 · Ecommerce Admin (Catálogo Web + Órdenes de Venta)

Objetivo: Publicar productos a la tienda online y gestionar órdenes de venta desde el panel admin.

6.1 — Queries y Actions modules/ecommerce/ (admin side)

TypeScript

// — queries.ts (admin) —

```
// fetchProductosWeb(filters)
SELECT pw.*, p.sku_base, p.nombre, p.activo AS producto_activo,
       m.nombre AS marca_nombre,
       (SELECT url FROM producto_imagenes pi
        WHERE pi.producto_id = p.id AND pi.es_principal = true LIMIT 1) AS
imagen,
       (SELECT COUNT(*) FROM variantes_producto vp
        WHERE vp.producto_id = p.id AND vp.activo = true) AS
variantes_count
FROM productos_web pw
JOIN productos p ON pw.producto_id = p.id
LEFT JOIN cat_marcas m ON p.marca_id = m.id
// Filtros: activo, en_oferta, destacado, nuevo, search (slug/sku/nombre)
ORDER BY pw.orden_display
```

```
// fetchProductosNoPublicados()
// Productos que NO tienen registro en productos_web:
SELECT p.id, p.sku_base, p.nombre, m.nombre AS marca
FROM productos p
LEFT JOIN cat_marcas m ON p.marca_id = m.id
LEFT JOIN productos_web pw ON pw.producto_id = p.id
WHERE pw.id IS NULL AND p.activo = true AND p.estado = 'publicado'
```

```
// fetchOrdenesVenta(filters)
SELECT ov.*
FROM ordenes_venta ov
ORDER BY ov.fecha_orden DESC
// Filtros: estado
('pendiente','procesando','enviado','entregado','cancelado'),
// rango fecha, search (numero_orden, email_cliente,
nombre_cliente)
```

```
// fetchOrdenVentaById(id)
// 1. Orden cabecera (ordenes_venta completa, incluye direccion_envio
JSONB)
// 2. Items:
SELECT oi.*, vp.sku_completo, p.nombre AS producto_nombre,
       t.codigo AS talla, c.nombre AS color,
       (SELECT url FROM producto_imagenes pi
        WHERE pi.producto_id = p.id AND pi.es_principal = true LIMIT 1) AS
imagen
FROM orden_items oi
JOIN variantes_producto vp ON oi.variante_id = vp.id
JOIN productos p ON vp.producto_id = p.id
LEFT JOIN cat_tallas t ON vp.talla_id = t.id
LEFT JOIN cat_colores c ON vp.color_id = c.id
WHERE oi.orden_id = ?
```

// — actions.ts (admin) —

```
// publicarProductoWeb(data)
INSERT INTO productos_web {
  producto_id, // FK → productos (UNIQUE implícito en práctica)
  slug, // text UNIQUE — generar: slugify(producto.nombre +
sku_base)
  titulo_seo,
  descripcion_seo,
  keywords,
  precio_publico, // numeric NOT NULL
```

```
precio_oferta, // numeric nullable
en_oferta, // boolean
destacado, // boolean
nuevo, // boolean
activo, // boolean
orden_display // integer
}
```

```
// actualizarProductoWeb(id, data)
UPDATE productos_web SET ... WHERE id = ?
```

```
// despublicarProducto(id)
UPDATE productos_web SET activo = false WHERE id = ?
```

```
// actualizarEstadoOrden(ordenId, nuevoEstado)
UPDATE ordenes_venta SET
  estado = nuevoEstado,
  updated_at = now(),
  -- Si estado = 'enviado':
  fecha_envio = CASE WHEN nuevoEstado = 'enviado' THEN now() ELSE
fecha_envio END,
  -- Si estado = 'entregado':
  fecha_entrega = CASE WHEN nuevoEstado = 'entregado' THEN now()
ELSE fecha_entrega END
WHERE id = ordenId
```

```
// actualizarRastreo(ordenId, numeroRastreo)
UPDATE ordenes_venta SET numero_rastreo = ? WHERE id = ?
```

```
// generarNotaSalidaPorOrden(ordenId, bodegaOrigenId, usuarioid)
// FLUJO AUTOMÁTICO al despachar:
// 1. Obtener items de la orden:
// SELECT oi.variante_id, oi.cantidad, vp.producto_id
// FROM orden_items oi
// JOIN variantes_producto vp ON oi.variante_id = vp.id
// WHERE oi.orden_id = ?
// 2. sp_crear_nota(tipo=SALIDA, bodega_origen=bodegaOrigenId)
// 3. Por cada item:
// sp_agregar_producto_nota(nota_id, cajas=0, producto_id,
piezas_sueltas=cantidad)
// ⚠ Nota: ecommerce vende por PIEZA (piezas_sueltas), no por caja
// 4. Opcionalmente confirmar nota automáticamente
```

6.2 — Gestión Catálogo Web app/(admin)/ecommerce/productos-web/page.tsx

text

Vista dividida en 2 secciones:

SECCIÓN A: Productos Publicados

DataTable:

Imagen	SKU	Slug	Precio Público	P. Oferta	Oferta?	Nuevo
Dest.	Act.					
dest.	sku_base	slug	precio_publico	precio_of.	en_oferta	nuevo
	activo					

Acciones: Editar, Despublicar, Ver en tienda (link a `/(store)/catalogo/[slug]`)

SECCIÓN B: Productos Disponibles para Publicar

Lista de productos con estado='publicado' que NO están en `productos_web`

Botón "Publicar" → abre modal con campos de `productos_web`

6.3 — Gestión Órdenes de Venta `app/(admin)/ecommerce/ordenes-venta/`

text

page.tsx — DataTable:

N° Orden	Cliente	Email	Total	Estado	Fecha	Rastreo
numero_orden	nombre_cliente	email_cliente	total	estado	fecha_	numero_
		(moneda)	(badge)	orden	rastreo	

[id]/page.tsx — Detalle:

CABECERA: datos cliente, dirección envío (parsear JSONB), estado, fechas

ITEMS:

Imagen	Producto	Talla	Color	SKU	Cant.	Subtotal

RESUMEN:

Subtotal: `ordenes_venta.subtotal`
Envío: `ordenes_venta.envio`
Impuestos: `ordenes_venta.impuestos`
TOTAL: `ordenes_venta.total`

ACCIONES:

Cambiar estado: pendiente → procesando → enviado → entregado
Agregar N° rastreo
● "Despachar" → genera nota de salida automática:
1. Seleccionar bodega de salida
2. Se genera nota con `sp_crear_nota` + `sp_agregar_producto_nota`
3. Confirmar nota → stock se descuenta
4. Estado orden → 'enviado'
Cancelar orden

6.4 — Tests

text

- ✓ Publicar producto → aparece en `productos_web` con slug correcto
- ✓ Editar precio público y oferta → verificar UPDATE
- ✓ Despublicar → `activo = false` → no aparece en store
- ✓ Orden de venta visible con todos sus items
- ✓ Despachar orden → genera nota de salida → confirmar → stock baja
- ✓ Agregar N° rastreo → verificar UPDATE
- ✓ Flujo completo: Publicar → Compra (Fase 7) → Orden aparece → Despachar → Stock baja

FASE 7 · Tienda Online (Ecommerce Público)

Objetivo: Storefront funcional con catálogo, detalle, carrito y checkout.

7.1 — Layout Store `app/(store)/layout.tsx`

text

Layout público (sin autenticación requerida):

```
<Navbar>
  Logo / Inicio
  Catálogo (link a /catalogo)
  Categorías (dropdown con cat_tipo_prenda WHERE activo AND
vista_web IS NOT NULL)
  Marcas (dropdown con cat_marcas WHERE activo)
```

 Carrito (ícono + badge con COUNT de carrito_items)

Login/Mi cuenta (si hay sesión → link a perfil, si no → link a /login)

```
</Navbar>
```

```
{children}
```

```
<Footer>
```

7.2 — Queries Store `modules/ecommerce/queries.ts` (store side)

TypeScript

// — fetchProductosWebPublicos(filters)

```
SELECT pw.id, pw.slug, pw.precio_publico, pw.precio_oferta,
  pw.en_oferta, pw.destacado, pw.nuevo,
  p.id AS producto_id, p.sku_base, p.nombre, p.descripcion,
  m.nombre AS marca,
  tp.nombre AS tipo_prenda, tp.vista_web,
  g.nombre AS genero,
  (SELECT url FROM producto_imagenes pi
  WHERE pi.producto_id = p.id AND pi.es_principal = true
  LIMIT 1) AS imagen_principal
FROM productos_web pw
```

```
JOIN productos p ON pw.producto_id = p.id
LEFT JOIN cat_marcas m ON p.marca_id = m.id
LEFT JOIN cat_tipo_prenda tp ON p.tipo_prenda_id = tp.id
LEFT JOIN cat_generos g ON p.genero_id = g.id
WHERE pw.activo = true
  AND p.activo = true
```

// Filtros públicos:

```
// tipo_prenda_id, marca_id, genero_id
// precio_min/max → pw.precio_publico BETWEEN
// en_oferta = true, destacado = true, nuevo = true
// search → p.nombre ILIKE
// ORDER BY pw.orden_display, pw.fecha_publicacion DESC
```

// — fetchProductoWebBySlug(slug)

// Producto completo para página de detalle:

// 1. productos_web + productos (todos los campos para SEO)

// 2. variantes_producto WHERE activo = true

// JOIN cat_tallas, cat_colores

// → para selector talla/color en UI

// 3. producto_imagenes ORDER BY orden

// → galería

// 4. producto_tags con JOIN tipo_tag, ref_tag

// → mostrar características

// 5. medidas_producto con JOIN cat_tallas, puntos_medida

// → tabla de medidas

// 6. Si es_conjunto → producto_conjunto con JOINS

// 7. Incrementar visitas:

// UPDATE productos_web SET visitas = visitas + 1 WHERE slug = ?

// — fetchProductosRelacionados(productoId, tipoPrendaId, marcaId)

// Productos del mismo tipo o marca:

```
SELECT pw.slug, pw.precio_publico, pw.precio_oferta, pw.en_oferta,
  p.nombre, (imagen_principal subquery)
FROM productos_web pw
```

```
JOIN productos p ON pw.producto_id = p.id
```

```
WHERE pw.activo = true
```

```
  AND pw.producto_id != productoId
```

```
  AND (p.tipo_prenda_id = tipoPrendaId OR p.marca_id = marcaId)
```

```
LIMIT 8
```

// — Carrito

// fetchCarrito(sessionId?, usuarioId?)

// Si hay usuario_id:

```
SELECT c.id, ci.id AS item_id, ci.variante_id, ci.cantidad,
  ci.precio_unitario,
```

```
  vp.sku_completo, p.nombre, p.id AS producto_id,
```

```
  t.codigo AS talla, col.nombre AS color, col.hex_code,
```

```

    pw.slug,
    (SELECT url FROM producto_imagenes pi
     WHERE pi.producto_id = p.id AND pi.es_principal = true LIMIT 1) AS
imagen
FROM carritos c
JOIN carrito_items ci ON ci.carrito_id = c.id
JOIN variantes_producto vp ON ci.variante_id = vp.id
JOIN productos p ON vp.producto_id = p.id
LEFT JOIN cat_tallas t ON vp.talla_id = t.id
LEFT JOIN cat_colores col ON vp.color_id = col.id
LEFT JOIN productos_web pw ON pw.producto_id = p.id
WHERE c.usuario_id = ? OR c.session_id = ?
ORDER BY ci.created_at

```

7.3 — Catálogo [app/\(store\)/catalogo/page.tsx](#)

text

Grid de <ProductCard> con:

- producto_imagenes (es_principal) → <Image>
- productos.nombre
- cat_marcas.nombre
- productos_web.precio_publico (formateado: \$1,299.00)
- productos_web.precio_oferta (tachado + precio normal si en_oferta)
- Badges: "Nuevo" (si nuevo=true), "Oferta" (si en_oferta=true)
- Link → /catalogo/{slug}

Sidebar de filtros:

- cat_tipo_prenda (checkboxes, solo WHERE vista_web IS NOT NULL)
- cat_marcas (checkboxes)
- cat_generos (checkboxes)
- Rango de precio (slider o inputs min/max)
- Toggle: Solo ofertas, Solo nuevos

Paginación o infinite scroll.

7.4 — Detalle Producto [app/\(store\)/catalogo/\[slug\]/page.tsx](#)

text

Server Component con generateMetadata() para SEO:

- title: productos_web.titulo_seo || productos.nombre
- description: productos_web.descripcion_seo || productos.descripcion
- keywords: productos_web.keywords

Layout de la página:

GALERÍA		INFORMACIÓN	
imagen principal [thumb][thumb][thumb] (producto_imagenes JOIN cat_tallas, ordenadas ORDER BY orden)		Marca: cat_marcas.nombre	
		Nombre: productos.nombre	
		Precio: \$1,299.00	
		(si en_oferta: precio_oferta tachado)	
		Talla: [S] [M] [L] [XL]	
		(de variantes_producto activas	
		JOIN cat_tallas, ordenadas)	
		Color: [●Negro] [●Azul]	
		(de variantes activas	
	JOIN cat_colores, con hex_code)		
		SKU: variante seleccionada.sku_comp	
		Cantidad: [1] [+][-]	
		[🛒 Agregar al carrito]	
		Composición: productos.composicion	
		Descripción: productos.descripcion	

TABS:

[Detalles] [Medidas] [Cuidado]
Detalles: producto_tags agrupados por tipo_tag.nombre Medidas: tabla medidas_producto (filas=puntos, cols=tallas) Cuidado: cat_telas.instrucciones_base_cuidado
PRODUCTOS RELACIONADOS
[card][card][card][card] (fetchProductosRelacionados)

Si es_conjunto = true:

Sección adicional "Este conjunto incluye:"

Lista de producto_conjunto.producto_hijo con imagen y link

7.5 — Carrito [hooks/useCarrito.ts](#) + páginas

TypeScript

// hook useCarrito:

// - Si hay sesión (usuario_id) → usa tabla carritos + carrito_items

// - Si no hay sesión → usa localStorage como buffer temporal

// - Al hacer login → merge: localStorage items → INSERT carrito_items → limpiar localStorage

// addToCart(variantId, cantidad, precioUnitario)

// 1. Obtener o crear carrito:

// INSERT INTO carritos (usuario_id | session_id) ... ON CONFLICT DO NOTHING

// 2. Agregar item:

// INSERT INTO carrito_items (carrito_id, variante_id, cantidad, precio_unitario)

// ON CONFLICT → UPDATE SET cantidad = cantidad + nueva_cantidad

// updateQuantity(itemId, nuevaCantidad)

// UPDATE carrito_items SET cantidad = ? WHERE id = ?

// Si cantidad = 0 → DELETE

// removeItem(itemId)

// DELETE FROM carrito_items WHERE id = ?

// clearCart(carritoId)

// DELETE FROM carrito_items WHERE carrito_id = ?

// CartDrawer (components/store/CartDrawer.tsx):

// Drawer lateral con lista de items, subtotal, botón "Ver carrito" y "Checkout"

// app/(store)/carrito/page.tsx:

// Tabla completa: imagen, producto, talla/color, precio, cantidad (editable), subtotal

// Botón eliminar por item

// Total del carrito

// [Seguir comprando] [Ir al checkout →]

7.6 — Checkout [app/\(store\)/checkout/page.tsx](#)

text

Formulario:

DATOS DE CONTACTO:

nombre_cliente (text, requerido)

email_cliente (email, requerido)

telefono_cliente (text)

DIRECCIÓN DE ENVÍO (se guarda como JSONB en ordenes_venta.direccion_envio):

```

{
  calle: string,
  numero_ext: string,
  numero_int?: string,

```

```
colonia: string,  
ciudad: string,  
estado: string,  
codigo_postal: string,  
pais: string (default "México"),  
referencias?: string  
}
```

RESUMEN DEL PEDIDO:

Items del carrito (solo lectura)
Subtotal
Envío (calculado o fijo por ahora)
Impuestos
TOTAL

NOTAS:

notas_cliente (textarea, opcional)

[Confirmar Pedido]

```
→ INSERT INTO ordenes_venta {  
  numero_orden,    // generar: "OV-" + timestamp o secuencial  
  usuario_id,      // nullable (guest checkout)  
  email_cliente,  
  nombre_cliente,  
  telefono_cliente,  
  direccion_envio, // JSONB  
  subtotal,  
  envio,  
  impuestos,  
  total,  
  estado: 'pendiente',  
  metodo_pago,     // por ahora null (integración pago en fase futura)  
  notas_cliente  
}  
→ INSERT INTO orden_items (por cada carrito_item) {  
  orden_id,  
  variante_id,  
  cantidad,  
  precio_unitario,  
  subtotal        // cantidad × precio_unitario  
}  
→ Vaciar carrito (DELETE carrito_items + DELETE carritos)  
→ Redirect a página de confirmación con numero_orden
```

7.7 — Tests

text

- ✓ Home muestra productos destacados (WHERE destacado=true)
 - ✓ Catálogo lista solo productos con productos_web.activo=true Y productos.activo=true
 - ✓ Filtros funcionan: por tipo_prenda, marca, genero, precio, ofertas
 - ✓ Detalle: slug resuelve correctamente → SEO metadata generada
 - ✓ Selector talla/color filtra variantes activas → muestra sku_completo
 - ✓ Galería de imágenes funciona ordenada
 - ✓ Tabla de medidas se renderiza correctamente con puntos_medida
 - ✓ Agregar al carrito (guest) → localStorage → ver en CartDrawer
 - ✓ Agregar al carrito (autenticado) → tabla carritos/carrito_items
 - ✓ Login con carrito guest → merge a carrito en DB
 - ✓ Checkout → crear orden → verificar ordenes_venta + orden_items en DB
 - ✓ Orden aparece en panel admin (Fase 6.3) → despachar → stock baja
-